

Running Software Research Programs: An Agile Approach

Janne Järvinen
F-Secure
Helsinki, Finland
janne.jarvinen@f-secure.com

Tua Huomo
VTT Technical Research Centre
of Finland
Oulu, Finland
tua.huomo@vtt.fi

Tommi Mikkonen
University of Helsinki and
Tampere University of Technology
Helsinki, Finland
tommi.mikkonen@cs.helsinki.fi

Abstract—Agile, lean processes have become the de-facto way to operate in the domain of software intensive products. Methodologies such as the lean startup are reshaping the way new companies and even well-established enterprises seek new opportunities in their operations. In contrast, in research, little has changed during that time – organizations that fund research still expect a solid, linear research plan. In this paper, we present an attempt to challenge this model in software research, based on 7 years of experiences in two large, national, industry-led projects that followed a more agile mindset. Furthermore, we also provide an insight to key learnings and best practices of running software research in agile fashion.

Keywords—Software research, agile, lean, experimentation

I. INTRODUCTION

Research programs require considerable up-front planning and promise for research to be done. For instance, EU programmes such as Frame Program 7 and Horizon 2020 [1] call for descriptions and specifications of hundreds of pages, with the end goal to describe collaborative research between academia and industry. The time window of these projects is extended over a number of years, with several millions of euros invested in each program. While there are research initiatives that can provide a good match for such a research approach, software research that aims at new abstractions, concepts, and approaches does not fall into this category. In such research, the context resembles the domain of software intensive products, where agile and lean development are the norm. Still, when it comes to software engineering research, little has changed over time – funding organizations still expect solid, waterfall model research proposals that are similar to shape and content as proposals were decades ago.

In this paper, we present an attempt to challenge the above model in software research, based on 7 years of experiences following a more agile mindset. The experience is gained from two national industry-led Finnish software research programs, Cloud Software [2] (2010-2013) and Need for Speed [3] (2014-2017), where numerous research institutes and companies have participated, with the total volume of 800 person years. The focus of this paper is clarify the enablers of agile execution of the program. We also provide an insight to key learnings and best practices of running software research in an agile fashion.

II. BACKGROUND AND MOTIVATION

Software research programs often are projects with large budgets and long time frames. With multiple partner orga-

nizations effective planning for the program is vital to its success. The shift from waterfall [4] to iterative, agile and lean development [5], [6], [7], [8], [9], [10] has taught us that facilitating change is more effective than attempting to prevent it [11]. We claim that rapid changes in the ICT industry have also created the need for more effective approaches for managing software research beyond traditional project management models.

Due to rapid changes in the field of ICT, the big paradigm changes are natural goals for joint research, as new ideas emerge. Recent examples of such include web and social media, cloud computing, big data, and continuous delivery, to mention a few examples. In our case, a determining factor where to go was based on industry needs, and the two paradigm changes we decided to go for were cloud computing [12] and DevOps style of operations [13].

Finally, generic, flexible funding models that support collaborative research are an important enabler for this type of an approach. In the end, they enable the collaborative set up and optimally drive the formation of critical mass as well as help creating established baselines for aspects such as intellectual property rights, consortium agreements, extending research to commercial collaboration, and so on.

III. TOWARDS LEAN RESEARCH APPROACH

The Lean Research Approach for Industry-driven Research (LRA) facilitates collaborative research between several organizations that have mutual interests in rapidly changing environment. The use of collaborative, long-term traditional project planning in some research fields remains useful. There is nothing predictable, however, about the rate of new innovations in the ICT sector except that the unexpected often happens faster than we expect. Yet there is typically a need to set the vision, goals and key results for the whole multi-year program as well as plan the collaboration between a number of industrial and research organizations.

In our case, the research goals are defined in Strategic Research and Innovation Agenda (SRIA) that serves as a roadmap for the research work [14]. Day-to-day research is organized in terms of business cases, or strategic research areas defined by industrial organisations, with business impact that is critical for future success. In addition, an element of experimentation can be included in each business case. The two key components of this approach are Continuous Planning and the Research Sprint model. Continuous planning

supports program's key goals through specific actions:

- Business case – here, strategically important research topic for an industrial organization – planning sets the focus on industry priorities and resource constraints.
- Self-organised way of working rather than project steering based on plans ensures that each party can focus on activities that are most essential for them.
- Transparency enables sharing information and results and advocates new innovation.
- Process discipline, transparency and accountability enhances the Program's access to assets and collaboration.

Research goals are defined at the beginning of every three month time period, called as the Research Sprint. At the beginning of the each quarter, the research team, with members from each participating organisation, defines a set of realistic tasks and outcomes over the next three months. Despite desperate initial claims “We can't make systematic, reliable research within three months!”, the Research Sprint approach has facilitated the division of the work effort and results in smaller, more manageable portions without compromising systematic and validated research methods.

Outcomes of Research Sprints vary. They may include, for example, design plans, methods descriptions, case studies, research articles, technical demonstrations, or business experiments. These artifacts are stored in Program's intranet – the main storage as well as access and sharing point for the Program's research assets. These assets are classified to the following 4 categories: 1) Software offerings (e.g. patents, copyrights); 2) Other Intellectual Property (e.g. product concepts, business models); Internal/Operational (e.g. new tools, processes, techniques); and 4) External/Relationship (e.g. business relationships, networks and ecosystems developed).

At the end of each sprint, teams presents the highlights of their work in a quarterly review meeting (Q-reviews). The main motivation behind the Q-reviews is to ensure transparency to achieve the attainment of best possible impact. The value of the continuous planning process is most visible in the Program's Q-review meetings where key results are presented and demonstrated at the Demo Bazaar. Each Q-review consists of themed workshops, training events, presentations, strategy discussions, and planning sessions.

A leadership troika has run each program. Focus Area Director (FAD) is responsible for the program execution, Academic Coordinator (ACO) commissions research activities, and Program Coordinator (PCO) coordinates program's operational work. In addition, the program has a steering group consisting of major industrial partners, and an extended steering group where academic partners are included. The work themes are divided into Communities of Practice (CoP) that consist of both industrial and academic partners.

Tools are an essential part of LRA as vast number of intermediate results are created. Tools also enhance collaboration through transparency – when you see what others have made you may come up with new ideas leading to even better results. As concrete tools, we have used Confluence [15] and Jira [16], as they both support the self-managed style of program execution we wanted to foster.

IV. EXPERIENCES

Both case projects were large industrial-driven research programs in Finland. They attracted a wide participation from the industry and academia as they hit significant turning points in ICT industry development. Cloud Software tackled the inception of the Cloud phenomenon and N4S coincided with even a more systemic transformation towards a world where digital resources are constantly available on-line, and available for all to use [17], [18]. Hence, the participants were diverse and they all shared common goals. Moreover, the core group of people have stayed together in both programs so the ways of working have been streamlined over the years. Yet several hundreds of people from tens of different organizations have participated in the collaboration so the experiences have been more than limited.

While the three-month research sprint remains at the core of operational work (1 and 6 month lengths were also tested) there has still been a need to review and fine-tune the overall program direction. Especially in the Cloud Software program, the first two years were very much research driven, while the last two years were driven by the intended industry impact with the introduction of Business Cases. In N4S program the rate of change was even faster as the Business Cases drove the program execution from the beginning.

There also has been difficulties and problems during the years. Funding cuts, company strategy changes and technology changes have all caused their concerns for the programs. While the collaborative and iterative way of working does not suit everybody the pros have clearly outweighed the cons. Most of the participants say that they do not want to go back to the traditional way of running collaborative projects spanning multiple organizations. In fact, in present-day research programs that are starting anew, the LRA model has been used by numerous participant organizations due to its ability to adapt to problems at hand on the fly.

V. CONCLUSIONS

Constantly changing environment and technological disruptions have created the need for more effective approaches to go beyond traditional project management models. In many fields, including software engineering in particular, this has been an established practice, whereas in software research, projects and programs have followed an approach, where everything is fully pre-planned.

Based on 7 years of experience on managing large software research programs, the adoption of new planning and project management approaches for their research efforts have allowed the named LRA programs to be ever more agile, collaborative and transparent. In addition, the case programs have been very successful in achieving their primary goal of developing enablers for globally successful businesses boosting their competitiveness, as well as creating high-quality research and publications at top forums co-authored across different organizations (e.g. [19], [20], [21], [22], [23], [24]). This perfect storm of research and engineering has inspired us to spread the practices also to other research programs both nationally and internationally.

REFERENCES

- [1] European Commission, “Horizon 2020,” <https://ec.europa.eu/programmes/horizon2020/>.
- [2] “Cloud Software Archives,” <http://activityblog.fi/tag/cloud-software/>.
- [3] “Need for Speed Program,” <http://www.n4s.fi/en/>.
- [4] W. W. Royce, “Managing the development of large software systems,” in *Proceedings of IEEE WESCON*, vol. 26, no. 8. Los Angeles, 1970, pp. 328–338.
- [5] B. W. Boehm, “A spiral model of software development and enhancement,” *Computer*, vol. 21, no. 5, pp. 61–72, 1988.
- [6] D. J. Anderson, *Kanban: successful evolutionary change for your technology business*. Blue Hole Press, 2010.
- [7] K. Beck, *Extreme programming explained: embrace change*. addison-wesley professional, 2000.
- [8] A. Cockburn, *Agile software development: the cooperative game*. Pearson Education, 2006.
- [9] E. Ries, *The lean startup: How today’s entrepreneurs use continuous innovation to create radically successful businesses*. Crown Business, 2011.
- [10] H. Terho, S. Suonsyrjä, K. Systä, and T. Mikkonen, “Understanding the relations between iterative cycles in software engineering,” in *Proceedings of the 50th Hawaii International Conference on System Sciences*, 2017.
- [11] C. Larman and V. R. Basili, “Iterative and incremental developments. a brief history,” *Computer*, vol. 36, no. 6, pp. 47–56, 2003.
- [12] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica *et al.*, “A view of cloud computing,” *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.
- [13] P. Debois, “Devops: A software revolution in the making,” *Journal of Information Technology Management*, vol. 24, no. 8, pp. 3–39, 2011.
- [14] T. Huomo, J. Järvinen, P. Kettunen, P. Kuvaja, A. Koivisto, C. Lassenius, P. Lehtovuori, S. Lilja, S. Miettinen, T. Mikkonen, J. Münch, T. Männistö, M. Oivo, J. Partanen, I. Porres, J. Still, and P. Tyräinen, “Strategic Research Agenda for Need for Speed,” <http://www.n4s.fi>, April 2015.
- [15] Atlassian, “Confluence,” <https://www.atlassian.com/software/confluence>.
- [16] —, “Jira,” <https://www.atlassian.com/software/jira>.
- [17] J. Järvinen, T. Huomo, T. Mikkonen, and P. Tyräinen, “From agile software development to mercury business,” in *International Conference of Software Business*. Springer, 2014, pp. 58–71.
- [18] N. P. Vitalari and H. Shaughnessy, *The Elastic Enterprise: the new manifesto for business revolution*, 2012.
- [19] N. Mäkitalo, J. Pääkkö, M. Raatikainen, V. Myllärniemi, T. Aaltonen, T. Leppänen, T. Männistö, and T. Mikkonen, “Social devices: collaborative co-located interactions in a mobile cloud,” in *Proceedings of the 11th International Conference on Mobile and Ubiquitous Multimedia*. ACM, 2012, p. 10.
- [20] M. Leppänen, S. Mäkinen, M. Pagels, V.-P. Eloranta, J. Itkonen, M. V. Mäntylä, and T. Männistö, “The highways and country roads to continuous deployment,” *IEEE Software*, vol. 32, no. 2, pp. 64–72, 2015.
- [21] M. Komssi, D. Pichlis, M. Raatikainen, K. Kindström, and J. Järvinen, “What are hackathons for?” *IEEE Software*, vol. 32, no. 5, pp. 60–67, 2015.
- [22] J. Lautamäki, A. Nieminen, J. Koskinen, T. Aho, T. Mikkonen, and M. Englund, “Cored: browser-based collaborative real-time editor for java web applications,” in *Proceedings of the ACM 2012 conference on Computer Supported Cooperative Work*. ACM, 2012, pp. 1307–1316.
- [23] M. Paasivaara, C. Lassenius, D. Damian, P. Rätty, and A. Schröter, “Teaching students global software engineering skills using distributed scrum,” in *Software Engineering (ICSE), 2013 35th International Conference on*. IEEE, 2013, pp. 1128–1137.
- [24] S. Mäkinen, M. Leppänen, T. Kilamo, A.-L. Mattila, E. Laukkanen, M. Pagels, and T. Männistö, “Improving the delivery cycle: A multiple-case study of the toolchains in finnish software intensive enterprises,” *Information and Software Technology*, vol. 80, pp. 175–194, 2016.